Fifteen Visualization and Interaction Techniques for Inspecting and Intervening in LLM Reasoning Within the Chat Interface

Visualizing Mechanistic Interpretability and Chain of Thought Reasoning

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science,
Virginia Tech

Abstract

Chat remains the interface where most people meet large language model reasoning, and it remains the interface least equipped to support scrutiny of that reasoning. Existing designs for visualizing chain-of-thought reasoning tend to move the reasoning out of the chat frame and into a canvas, a graph, or a separate replay view, which leaves the everyday exchange untouched. This paper proposes fifteen interaction techniques that operate inside the chat frame itself. Each technique addresses a specific failure: reliance on the model's self-report as the sole trust signal, alternatives that exist only as labels the model never traversed, commitments that dissolve between turns, and inspection that arrives after generation completes and so cannot change the answer. One principle organizes the set. Interfaces that allocate operator attention by uncertainty alone misallocate it; attention should track uncertainty weighted by downstream consequence, since a low-certainty claim that nothing depends on warrants less scrutiny than a moderate-certainty assumption carrying five decisions. For each technique the paper gives the visual encoding, the interaction, the layer of model access it requires, the dependent variable it makes measurable, and the closest prior art in the research literature and in shipping software. Five of the fifteen exist in recognizable form in research prototypes or products; five have partial precedents that begin to explore this visualization space; one adapts a standard offline analysis method that has no user-facing implementation; and four appear to be novel techniques with no precedent in either literature or product. The set forms a design space; several techniques compete for the same visual channel, and several consume additional inference, so the paper closes with the cost and encoding constraints that any implementation must resolve.

1. Introduction

Interfaces for large language model reasoning have converged on a small number of shapes. The reasoning trace appears in a collapsible panel above the answer, or as a node-link graph in a dedicated workspace, or as a replayable timeline the user scrubs after the fact. Each shape assumes the user will leave the conversation to inspect the reasoning. Most users do not leave. They read the answer in the chat frame, form a judgment there, and continue. **[TODO: cite a reference for this]**

That gap motivates the techniques below. All fifteen operate in the chat frame, on or immediately adjacent to the response text, and all of them assume the user is reading rather than investigating. The design target is Prototype A and Prototype B from the author's whitepaper entitled "Interactive Visual Chain-of-Thought: Designing Collaborative Reasoning Workspaces for Human-AI Sensemaking"; the canvas and the desktop environment stay out of scope.

The techniques attack four problems, and the paper groups them accordingly.

The first problem is the provenance of the trust signal. An emitted reasoning trace is one signal about the model's process, and Turpin et al. showed that traces can be systematically unfaithful to the computation that produced the answer [1]. The author's prior work concedes this point, then proceeds to encode the model's own certainty numbers anyway. Techniques 1 through 3 draw signal from other layers of access.

The second problem is the depth-one tree. Asking a model to name the alternatives it considered produces alternatives it never traversed, which is why generated counterfactual branches read as placeholders. Sampling traverses them. Techniques 4 and 5 replace named alternatives with sampled ones.

The third problem is amnesia across turns. Chat forgets its own commitments. The author's four prototypes visualize a single exchange or replay a completed history; none maintains a live contract that persists as the conversation advances. Techniques 6 through 9 maintain session state.

The fourth problem is timing. The introduction to the prior work promises that a user can correct a flawed premise mid-stream, and all four prototypes operate after generation completes. Techniques 10 through 12 move the intervention earlier, before or during generation. Techniques 13 through 15 then handle cross-layer traversal, the operation the six-layer access model implies but that no chat interface currently supports. **[TODO: cite the layered-access paper; align the layer numbers used below (Layer 2 for token distributions, Layer 4 for agent actions) with its published numbering]**

Each entry in the catalog closes with the closest prior art I could locate, drawn from the HCI and visualization literature, from the NLP methods literature, and from shipping software. Two findings recur and shape the priority argument in Section 3. First, several of these techniques rest on methods the NLP community treats as routine, such as context ablation and repeated sampling with semantic clustering, that no chat interface exposes to a user; the gap is in the interface, not in the method. Second, the techniques that already exist tend to exist in developer tools and coding assistants rather than in general chat, which means the design questions have been answered for programmers and left open for everyone else.

1.1 Consequence-Weighted Salience

One principle runs through the set and deserves statement before the catalog. Uncertainty visualization in reasoning interfaces allocates visual salience in proportion to uncertainty: the less certain a claim, the louder the encoding. That allocation is incomplete. A claim's demand on operator attention is a function of two quantities, its uncertainty and the number and weight of the conclusions that rest on it.

Consider two claims in the same response. The first carries 40 percent certainty and appears in a parenthetical aside that no later step references. The second carries 70 percent certainty and supports five downstream decisions, including the recommendation the user will act on. Uncertainty-proportional encoding makes the first louder. The second is where the user's scrutiny pays.

One empirical result supports the argument directly. Vasconcelos et al. compared three conditions in a preregistered study of thirty programmers: no highlighting, highlighting the tokens least likely to be generated, and highlighting the tokens most likely to be edited [2]. Highlighting by generation probability produced no benefit over no highlighting at all. Highlighting by predicted edit likelihood produced faster task completion and more targeted edits, and participants preferred it. Edit likelihood is a proxy for consequence: it predicts where the user will act. The finding is that a signal about the model's uncertainty, presented alone, did not help, and a signal about the user's prospective action did. Buçinca et al. reach a compatible conclusion from the other direction, showing that interventions which force engagement at the decision point reduce overreliance while simply presenting explanations does not [3].

Computing the weight requires a dependency structure over the response, which the reasoning graph already provides: count the descendants of a node, weight them by their own consequence, and multiply by the node's uncertainty. Several techniques below rely on this quantity, and Technique 3 measures it empirically instead of deriving it from the graph. **[TODO: settle whether consequence weight comes from graph topology, from the knockout probe, or from both; the chat did not resolve this]**

2. Design Space

The fifteen techniques appear below in five groups. Each entry states the visual form, the interaction, the layer of access required, what the technique contributes beyond the author's existing designs, and where the technique or its nearest relative already appears.

Table 1 summarizes the survey. *Deployed* marks a technique that exists in recognizable form in a research prototype or a product, though not always inside a chat frame. *Partial* marks a technique with a precedent that stops short of the operation described here. *Method only* marks a technique whose underlying computation is standard in the NLP literature and has no user-facing implementation. *Novel* marks a technique for which I found no precedent.

#	Technique	Status	Closest prior art
1	Hesitation Ribbon	Deployed	GLTR [4], HiLDe [5], vendor logprob views [6]
2	Cross-Layer Agreement Gutter	Novel	None located
3	Knockout Probe	Method only	ContextCite [7], ERASER metrics [8]
4	Sampled Path Distribution	Deployed, separate workspace	Self-consistency [9], semantic entropy [10], ChainForge [11], Loom [12]
5	Divergence Point Marker	Novel	Loom branch points [12]
6	Assumption Ledger	Partial	OnGoal [13], Kiro specs [14]
7	Open Questions Queue	Partial	Clarifying-question work [15], [16]; Kiro [14]
8	Reasoning Diff Between Turns	Novel	LMdiff [17] and LLM Comparator [18] compare models, not turns
9	Conversation Spine	Deployed, message-keyed	Sensecape [20], chat minimap extensions
10	Frame Card	Deployed in coding tools	Kiro [14], agent plan modes
11	Mid-Stream Intervention Window	Partial	HiLDe [5] at the token, AGDebugger [19] at the message
12	Certainty Threshold Rendering	Novel	Progressive disclosure [22]
13	Agent Action Ribbon	Deployed	AgentLens [23], AGDebugger [19], deep research activity panels
14	Provenance Descent	Partial	Citation drill-down in Perplexity and NotebookLM; details on demand [24]
15	Second-Opinion Overlay	Partial	Multiagent debate [25], LLM Comparator [18]

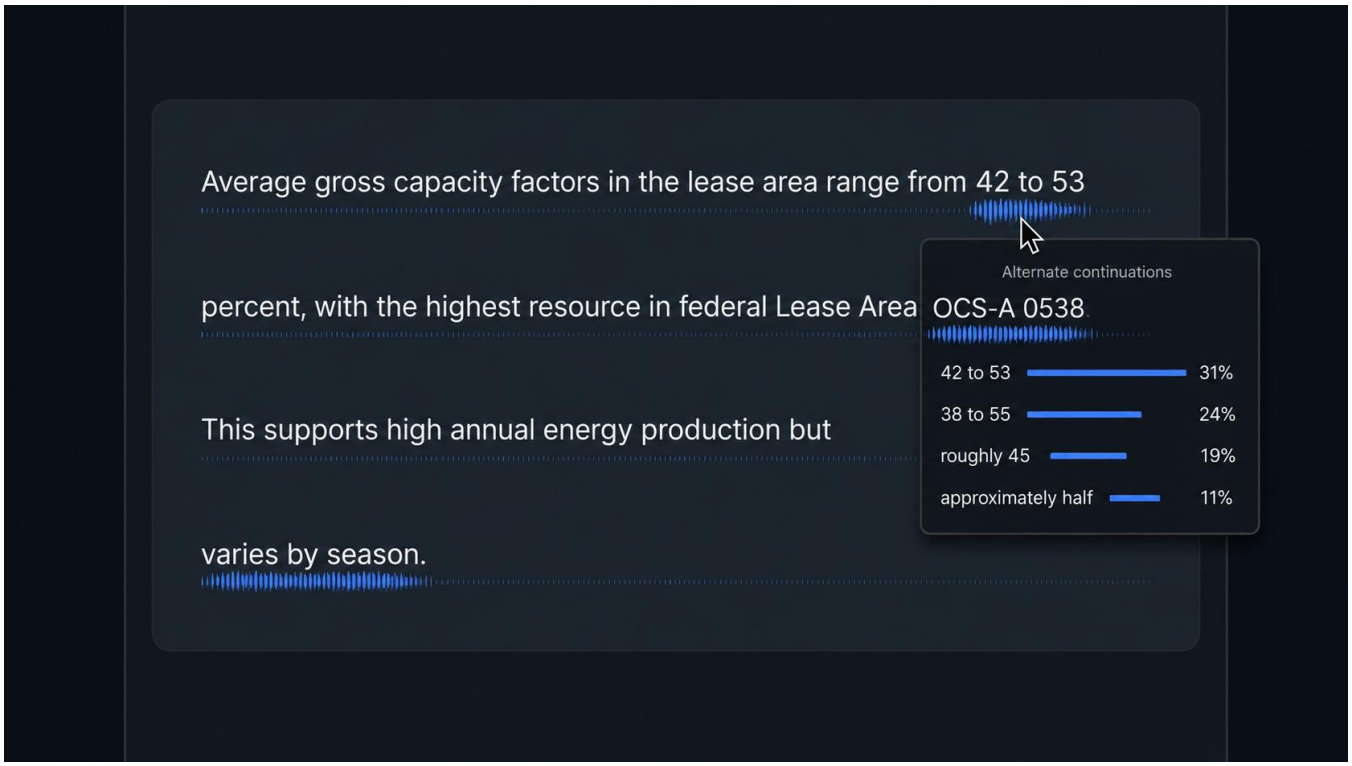
Table 1. Deployment status of the fifteen techniques.

2.1 Signals That Are Not the Model's Self-Report

The three techniques in this group draw on layers the hosted API does not expose. Each depends on the self-hosted inference deployment, which turns an infrastructure decision into a research asset. **[TODO: confirm the deployment details; the chat referenced a local vLLM deployment and a GLM-5.2 plan]**

1. Hesitation Ribbon

A thin ribbon runs under each line of response text, its density driven by per-token entropy read from the local inference server. A flat ribbon marks spans where the model committed. A frayed ribbon marks spans where the next token was contested. The user hovers a frayed span and reads the top-k alternates with their probabilities.



The encoding sits below the text baseline and consumes no horizontal space, which matters because the chat frame has little to spare. Density carries the signal, which leaves color free for the certainty encoding already in use.

The research payload is the divergence case. High self-reported certainty over a high-entropy span is a measurable disagreement between what the model says about its confidence and what its output distribution shows. The interface can flag those spans as confident but unstable, which gives the user a signal the model did not choose to emit. That comparison requires token-level access and so is unavailable to anyone building on a hosted API.

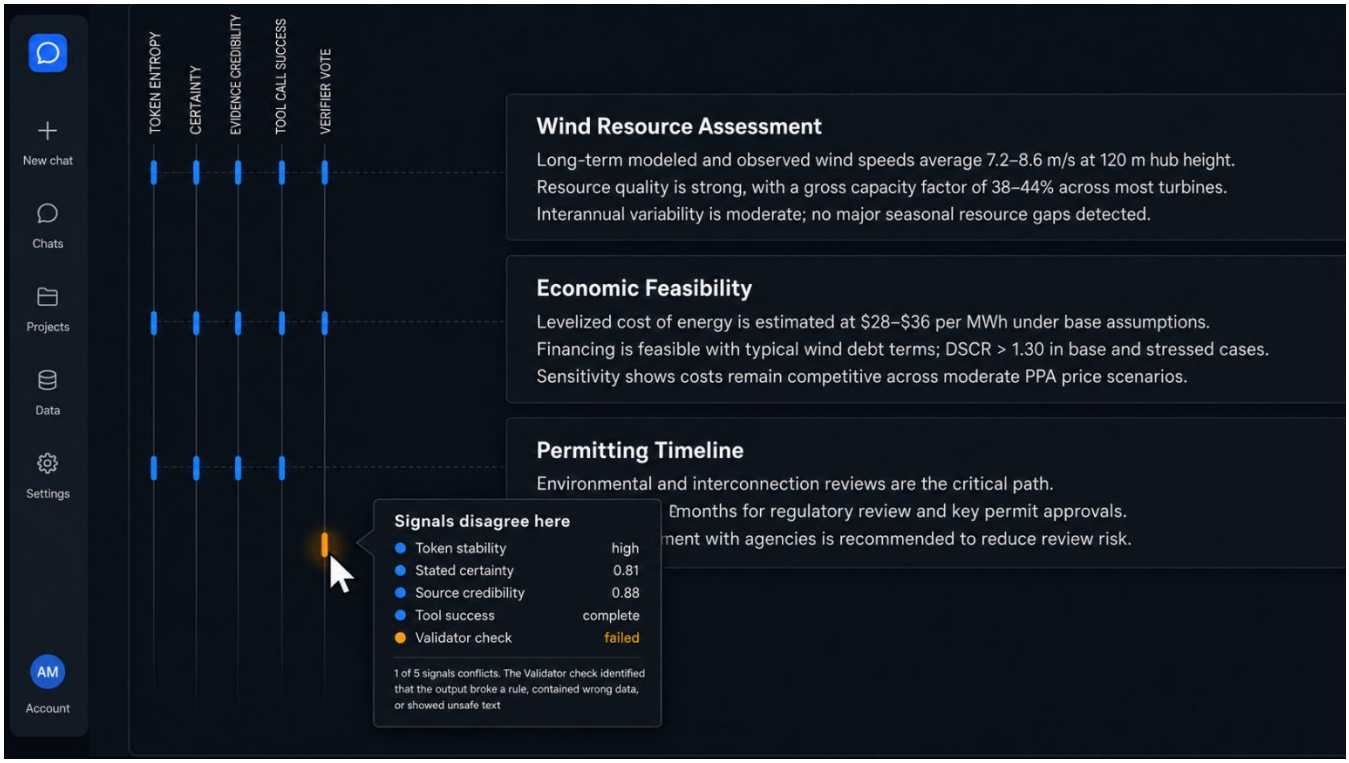
Token-level uncertainty highlighting is the most heavily precedented technique in this catalog. GLTR colored each token by its rank in the model's predicted distribution and raised human detection of generated text from 54 to 72 percent without training [4]. Vasconcelos et al. tested the encoding on AI code completion and found that highlighting by generation probability gave no benefit over no highlighting, a negative result worth planning around [2]. HiLDe, a Visual Studio Code extension, shades uncertain tokens in a red gradient, opens the alternates the model considered at that position, explains how each differs, and regenerates the remainder from the user's substitution [5]. On the API side, OpenAI, Ollama, and LM Studio all return per-token log probabilities with top-k alternates, and LM Studio and several playgrounds render them as colored text with click-to-expand alternates [6].

Two properties separate this design from that work. The divergence comparison, emitted certainty against token entropy over the same span, does not appear in any of it; every prior implementation encodes one quantity. Each of those implementations also spends color, which the chat frame has already committed to certainty; the ribbon spends density instead. Vasconcelos et

al. also report that programmers wanted highlights that were granular, interpretable, and not overwhelming [2], which argues for the ribbon's position below the baseline rather than a background wash over the text.

2. Cross-Layer Agreement Gutter

A vertical strip beside each response section holds one mark per layer of access: token entropy, emitted certainty, evidence credibility, tool-call success, verifier vote. Agreeing layers align into a clean column. Disagreement fractures the column, and the fracture is visible in peripheral vision without reading. The user clicks a fracture and gets a side-by-side account of what each layer reports about that passage.



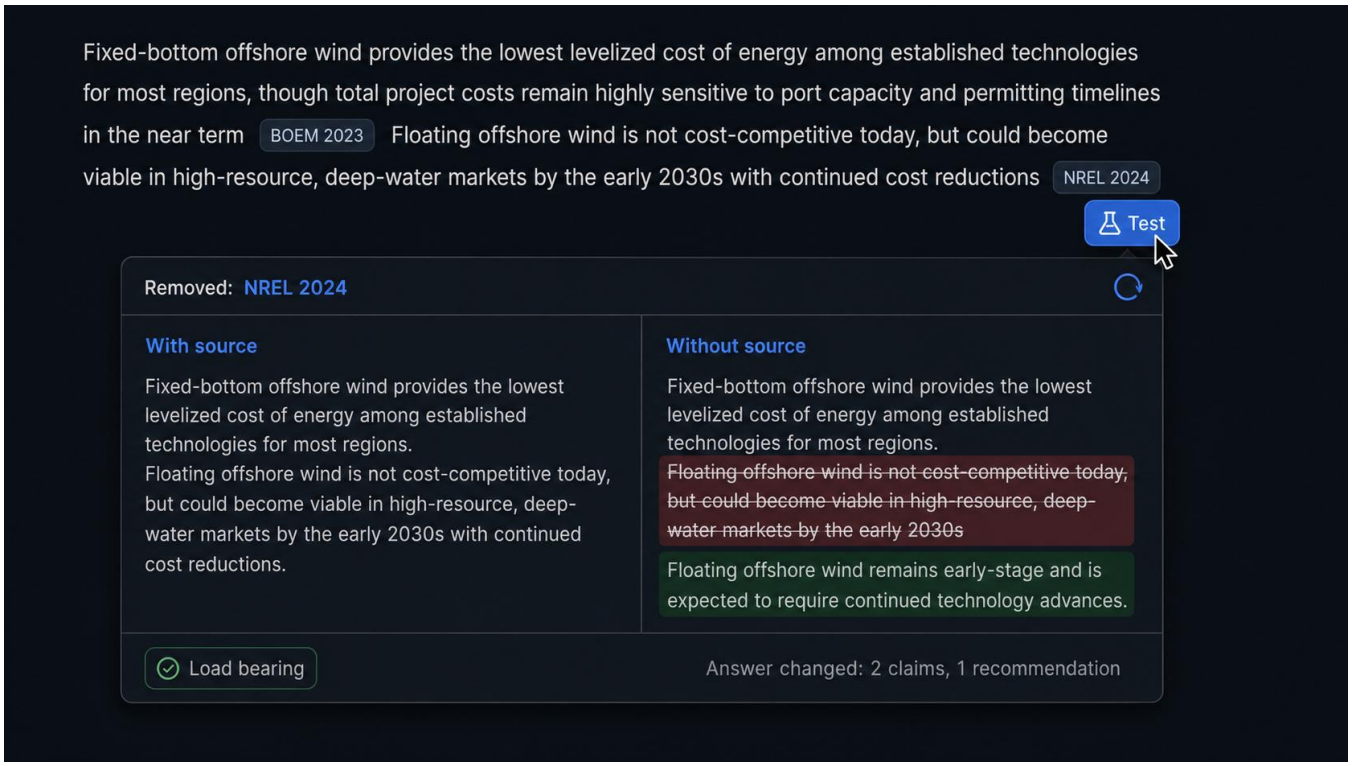
This technique answers an open question in Section 5 of the layered-access paper by making the interface the comparison instrument. Rather than waiting on a separate methodology for cross-layer comparison, the gutter performs the comparison continuously and surfaces the disagreements as they occur. **[TODO: verify the section number and restate the open question in the paper's own terms]**

The design risk is legibility at small sizes. Five marks in a strip narrow enough to sit in a chat margin gives each mark a few pixels, so the encoding must survive that budget. A pilot should test whether users detect fractures at realistic reading speeds before the technique earns a full study.

I found no system that renders per-layer agreement as a spatial column beside response text. The pieces exist separately. Uncertainty visualization research has studied single-signal encodings and their effect on reliance [2], [3]. Evaluation tools align two signals for offline analysis: LLM Comparator places two models' responses side by side with an automatic judge's verdict [18], and LMdiff aligns two models' token distributions [17]. Both target a developer inspecting a batch of results after the fact. The gutter proposes the same comparison, continuous, in the margin, for a reader in a conversation, and it appears to be novel.

3. Knockout Probe for Load-Bearing Evidence

For any cited evidence item, the interface re-asks the question with that item removed from context and diffs the resulting answer. Two outcomes follow. The answer changed, so the citation carried weight. The answer held, so the citation was decoration.



The user clicks a citation and presses "test this." A batch mode marks every citation in one response, which is the mode a user reviewing a long analysis will want. Results render as a badge on each citation, and the badge persists for the rest of the session.

This technique converts the faithfulness literature into an interaction. Leave-one-out ablation is a standard analysis method; placing it under the user's finger at the point of doubt makes it an interface affordance. It also produces a clean dependent variable: the proportion of cited evidence that proves load-bearing, measurable per response and comparable across conditions.

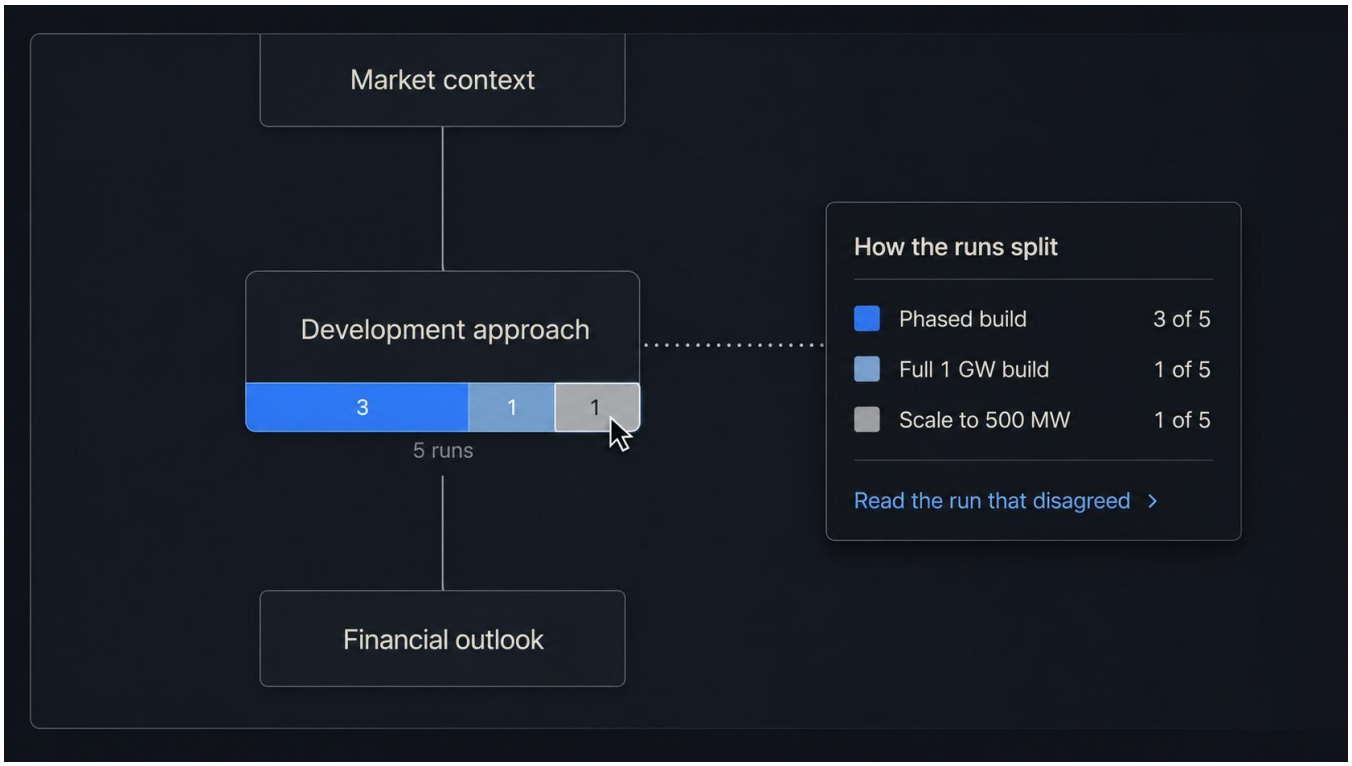
The method is settled and the interaction is missing. ContextCite formalizes context attribution, ablates context sources at random, fits a sparse linear surrogate over the resulting probability shifts, and demonstrates the result for verifying generated statements and detecting context poisoning [7]. The comprehensiveness and sufficiency metrics in ERASER apply the same leave-one-out logic to rationale evaluation [8]. Recent work on citation faithfulness uses ablation as a training signal rather than as a display. In all of it, the ablation runs offline and reports a number to a researcher. No product exposes the operation to the person reading the citation, and the design questions the interface raises, whether the verdict should persist for the session, whether batch mode should be the default on long analyses, and how to report a partial change rather than a binary, are open.

The cost is one additional inference per probe. Section 3.2 returns to this.

2.2 Making Alternatives Real

4. Sampled Path Distribution at Decision Nodes

Run the same prompt five times, cluster the resulting decisions, and render the split at each decision node as a small stacked bar. Four samples chose phased development; one chose scaling down to 500 MW. Alternatives now carry counts rather than labels.



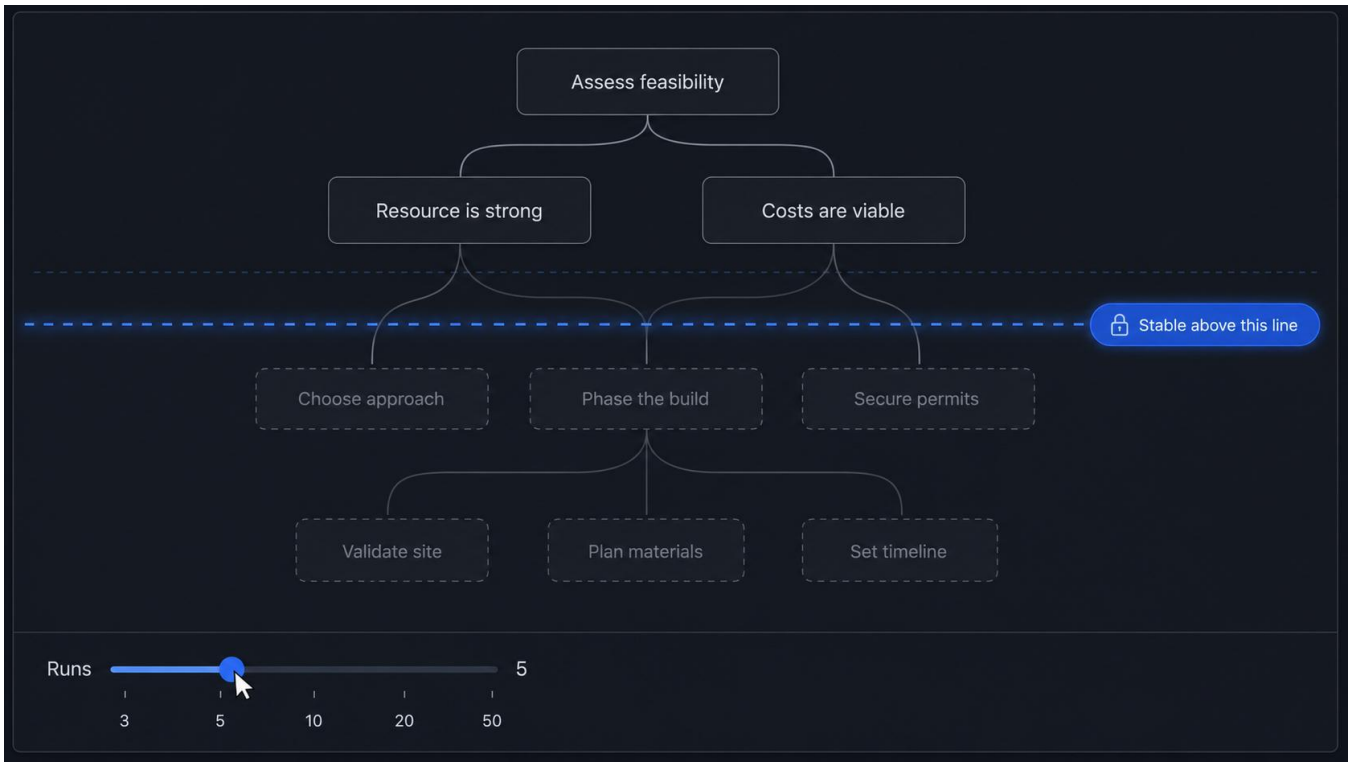
The user clicks the minority slice and reads the reasoning of the run that disagreed. That run produced a competing rationale by traversing it, so the user gets an argument instead of a name. The interaction is cheap for the user and expensive for the system, which reverses the usual arrangement in reasoning interfaces and is worth stating as a design position.

Clustering is the hard part. Five runs produce five phrasings of what may or may not be the same decision, and the clustering step must decide. Semantic clustering with a small embedding model is the obvious approach; the failure mode is over-merging, which hides genuine disagreement behind a unanimous-looking bar. Farquhar et al. supply a validated procedure for the same problem: they cluster sampled generations by bidirectional entailment, so two outputs join a cluster when each entails the other, and compute entropy over the clusters rather than over token sequences [10]. Adopting entailment-based clustering borrows a method with published discrimination results instead of inventing one. **[TODO: decide between entailment-based and embedding-based clustering and pilot the over-merging rate on your own decision-node data]**

The sampling method is standard and two tools already expose the distribution. Self-consistency samples multiple reasoning paths and marginalizes over them, and it is the reason resampling counts as a measurement rather than a gimmick [9]. ChainForge lets a user run a prompt many times across models and settings and inspect the spread of responses, and its authors report users doing exactly the opportunistic-then-systematic exploration this technique assumes [11]. Loom renders sampled continuations as a navigable tree the user prunes and extends [12]. Both keep the distribution in a workspace the user visits deliberately. Rendering the split at a decision node as a stacked bar inside the response, so that the count arrives unrequested and the minority run is one click away, is the contribution.

5. Divergence Point Marker

Across a set of samples, mark the earliest step where the runs split. Everything upstream of that point the model treats as settled. Everything downstream is contingent on a choice the model made without stable grounds. The marker renders as a horizontal line across the reasoning graph, labeled to indicate that reasoning above it survives resampling.



The user drags the sample count up or down and watches the line move. At two samples the line sits low, since two runs agree by chance for longer. At twenty it rises, and where it settles tells the user how much of the answer holds under resampling and how much is one path through a wide distribution.

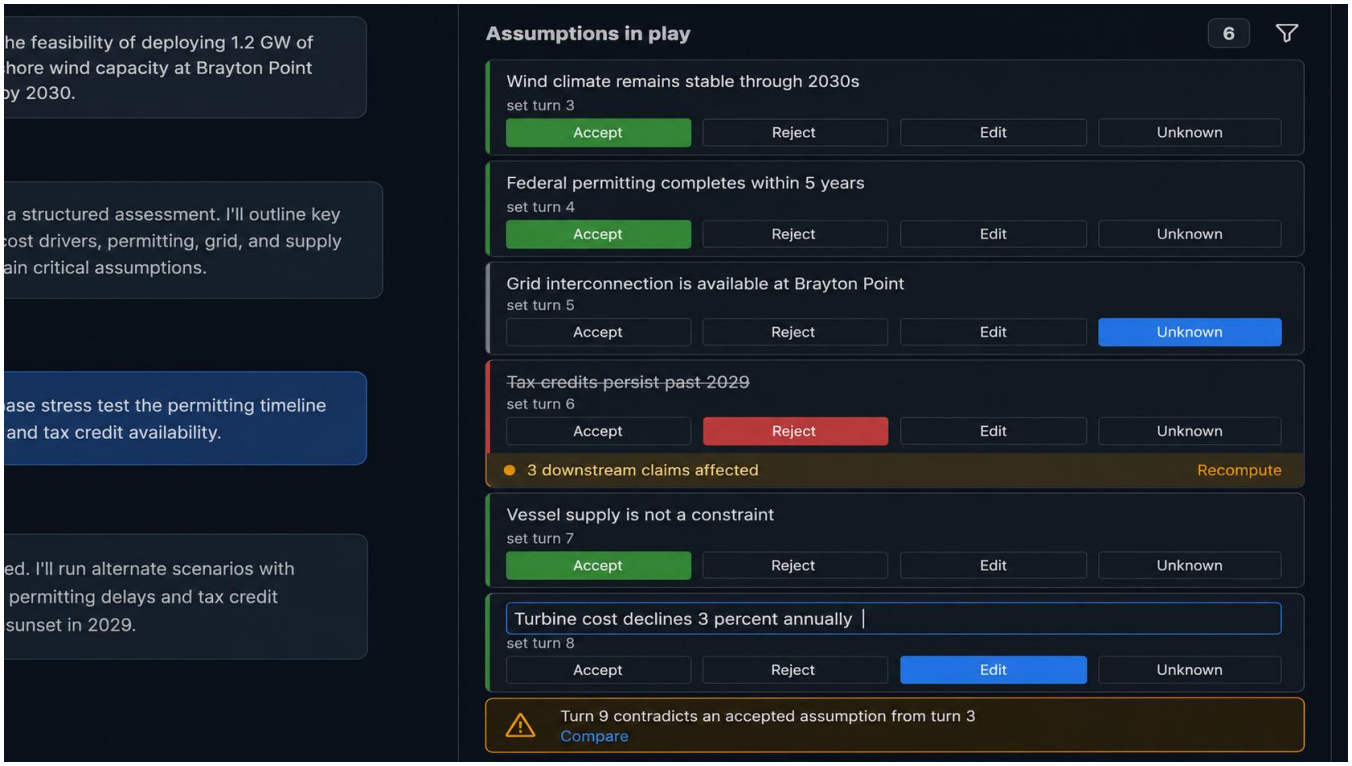
The interaction gives the user a direct manipulation over a statistical property, which is rare in reasoning interfaces and easy to explain. The line's position is also a natural dependent measure for a study of trust calibration: users who see a low line should discount the answer more, and whether they do is testable.

Loom is the closest relative: its tree view shows where sampled continuations fork, so a user can see the branch point [12]. The difference is what the display reports. Loom shows the branches a user chose to generate; this marker reports the earliest point at which a sample set stops agreeing, aggregated across runs, as a single line whose position responds to the sample count. I found no interface, research or commercial, that computes and displays that quantity. Treating it as a dependent measure of trust calibration also appears to be new.

2.3 State That Persists Across Turns

6. Assumption Ledger

A persistent panel lists every assumption the conversation currently rests on, accumulated across turns. Each entry carries four states: accepted, rejected, edited, unknown. Rejecting an assumption greys out the downstream claims that depend on it and offers a recompute.



Drift detection is the valuable case. Turn nine contradicts an assumption the user accepted at turn three, and the ledger flags the conflict and names both turns. Neither the user nor the model tracks that today, and the failure is quiet: the conversation proceeds on a foundation the user believes is fixed. The author's editable assumption cards handle a single response; the ledger handles the session.

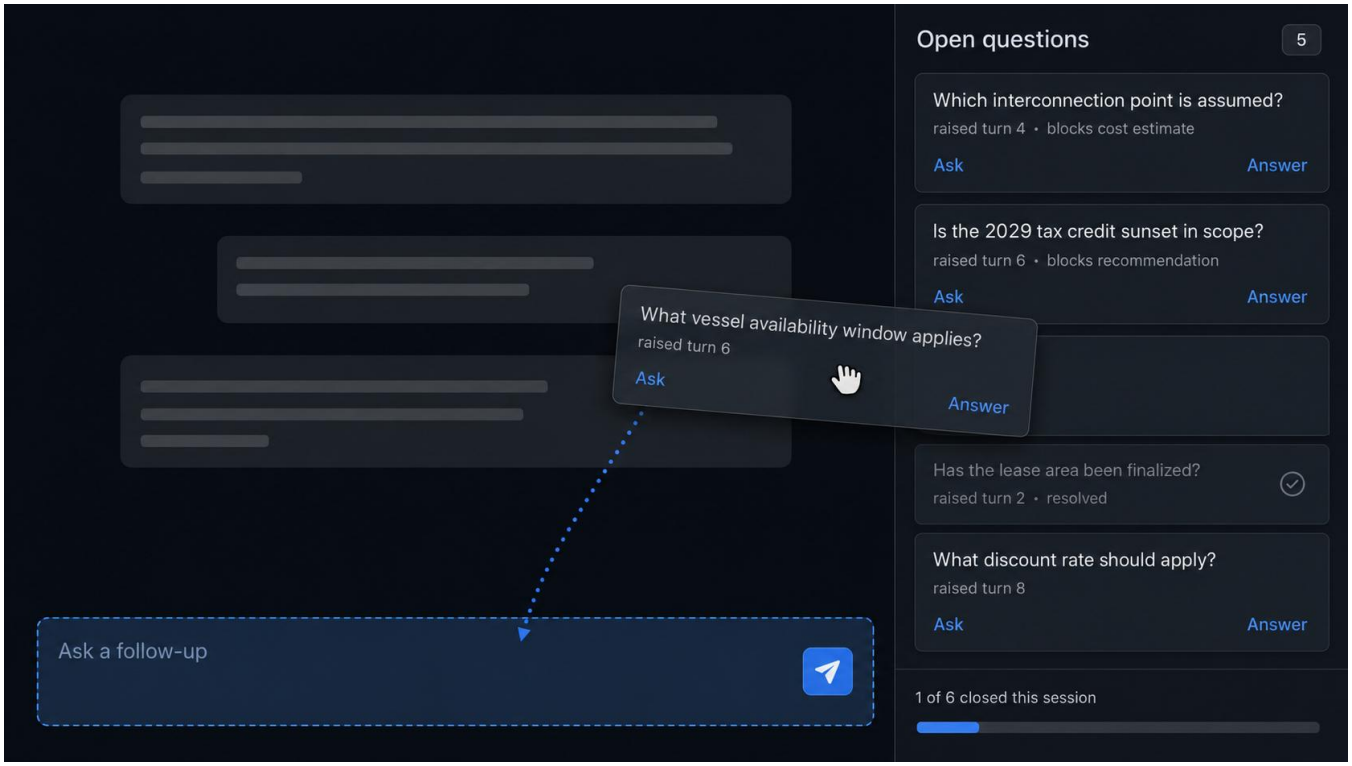
Implementation requires the model to emit assumptions in a structured form and requires the interface to match new assumptions against the existing set. Matching is the same clustering problem Technique 4 raises, at a different granularity.

One research system and one product come close. OnGoal augments a chat interface with goal tracking: it infers the user's goals, merges new goals against the existing set, evaluates each response against them, and renders progress both inline and in side summary views [13]. In a 20-participant writing study, participants using OnGoal reached their goals with less time and effort than a baseline chat interface, which is direct evidence that persistent session state pays. Kiro externalizes requirements into files the user approves before the model implements anything, and its requirements analysis pass reasons across the whole requirement set to flag logical inconsistencies, ambiguities, and conflicting constraints [14]. The merge problem this technique faces is the merge problem OnGoal solved for goals, and the conflict-detection problem is the one Kiro solved for requirements.

Two differences remain. OnGoal tracks what the user asked for; the ledger tracks what the model assumed, which the user never stated and may not have noticed. Kiro runs its consistency check on a static document on demand, while the ledger has to run it incrementally as each turn arrives. Drift detection across turns, where turn nine contradicts an assumption accepted at turn three and the interface names both turns, appears to be novel.

7. Open Questions Queue

The model's unresolved uncertainties collect in a sidebar as an explicit list rather than dissolving into hedged prose. Each entry carries the turn it arose in and what it blocks. The user clicks an entry to send it as the next prompt, or answers it inline, which converts it into a resolved constraint in the ledger.



The queue makes visible something that current interfaces bury. A model that writes "assuming the deployment is on-premises" has raised a question and answered it in the same clause; the user reading quickly absorbs the assumption without registering that a question was open. Extracting these into a list separates the raising from the answering.

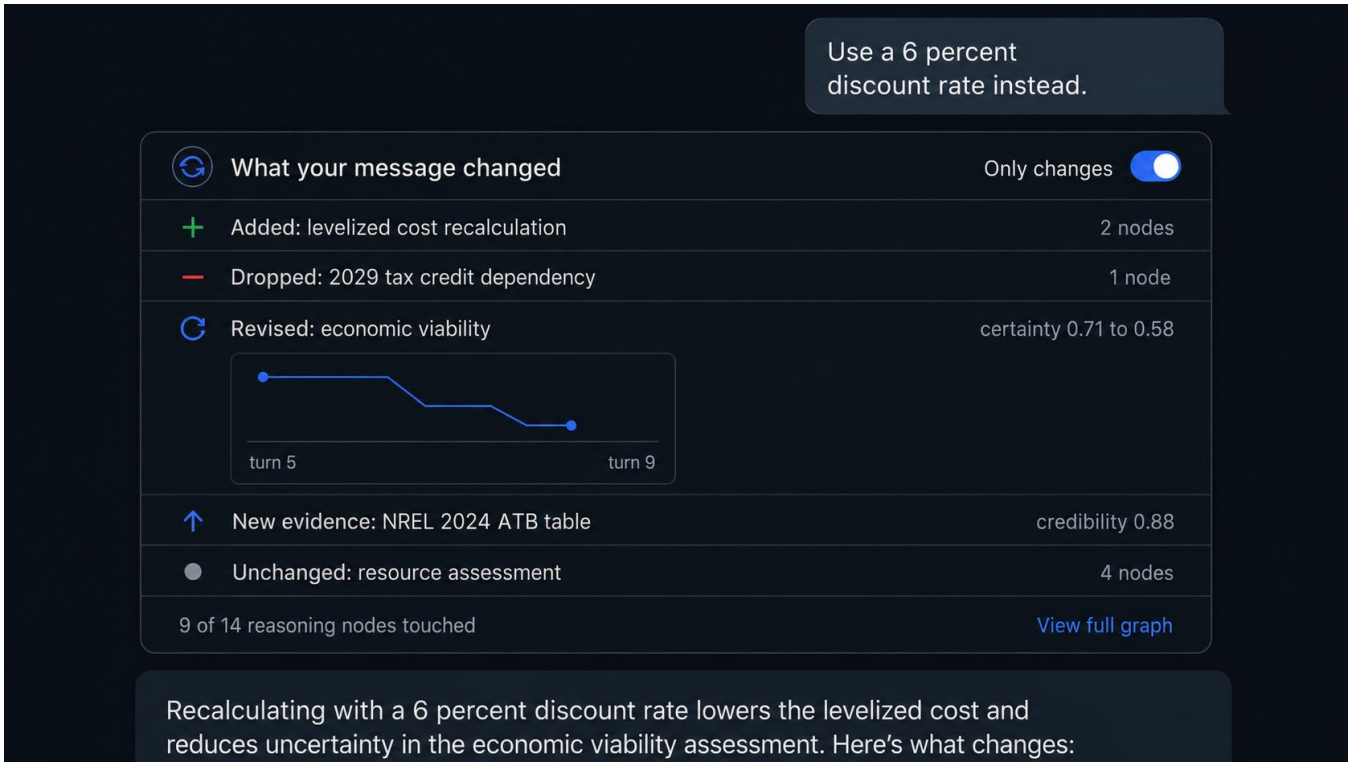
The technique also measures something directly. Count the open loops a user closes with the queue visible against a control condition with the queue hidden. That comparison needs no instrumentation beyond the log.

Clarifying questions have a long research record and a growing product presence, and in both cases they arrive at the front of the interaction and then disappear. Aliannejadi et al. framed clarifying-question selection for open-domain information seeking and released the supporting dataset [15]. Zhang et al. trained models to ask them by assigning preference labels from simulated outcomes in future turns, addressing the tendency of instruction-tuned models to presuppose one reading of an ambiguous request [16]. Kiro asks clarifying questions about scope, constraints, and edge cases before generating a spec [14], and the deep research modes in ChatGPT and Gemini confirm scope before starting a run.

Every one of those treats a question as a blocking prompt: the system asks, the user answers, the question is gone. This technique inverts the lifecycle. Questions accumulate rather than block, they carry the turn they arose in and what they block, and the user chooses when to spend attention on them. A persistent queue of the model's unresolved uncertainties has no precedent I located.

8. Reasoning Diff Between Turns

After each new user message, an inline card reports what changed in the reasoning structure: nodes added, nodes invalidated, certainty shifts, evidence that entered or fell out of consideration. The user toggles a filter to show only what the last message changed.

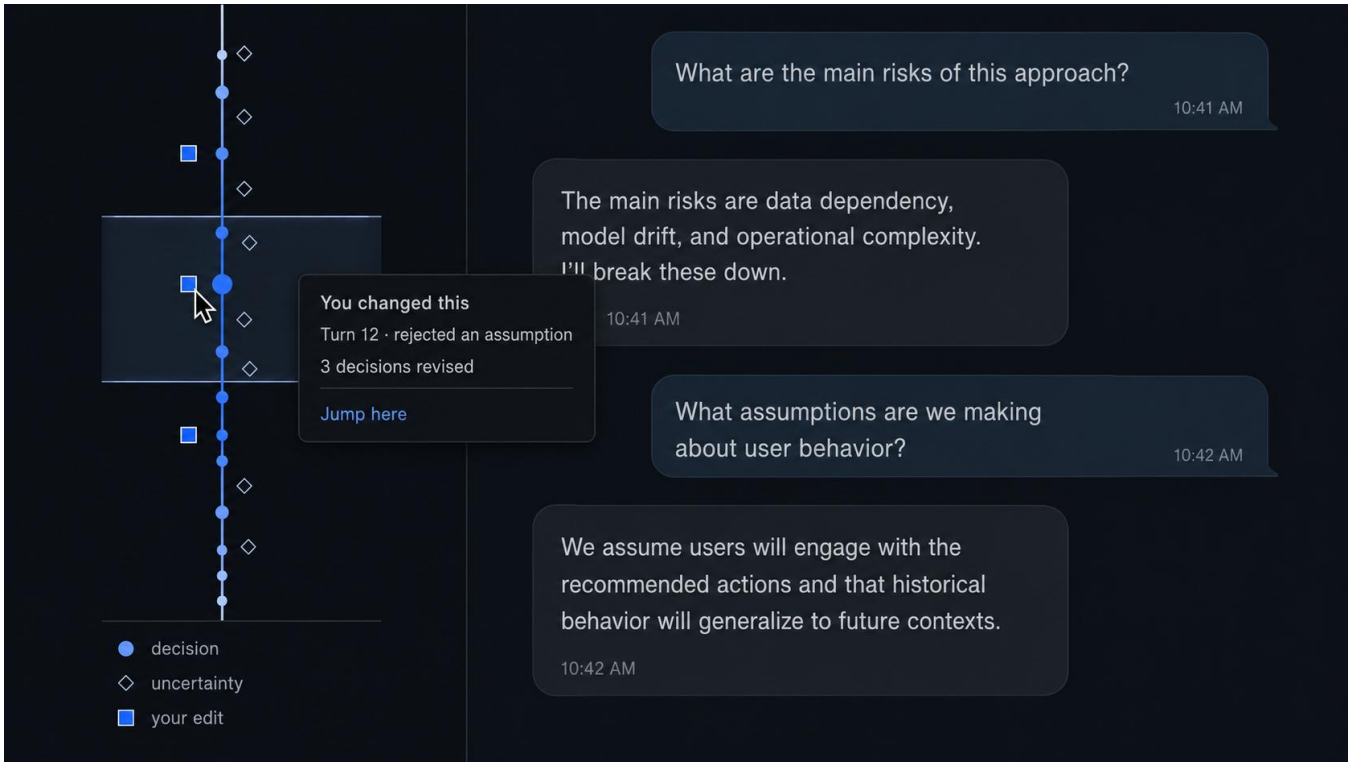


Navigation is the obvious benefit. The second benefit is instructional. A user who sees which nodes a rephrasing moved learns which phrasings change the reasoning and which only change the wording of the answer, which connects the technique to the prompt taxonomy work. **[TODO: cite the prompt-taxonomy paper]** Prompt engineering advice circulates as folklore because users get no feedback signal beyond the final text; the diff supplies one.

Diff tools for language models compare models rather than turns. LMdiff aligns the token-level probability distributions of two models to help a researcher find contexts where they diverge [17]. LLM Comparator compares two models' responses across a prompt set and helps the user characterize when and why one does better [18]. AGDebugger gives something closer to a turn diff by letting a developer reset to an earlier message, edit it, and rerun, but the comparison happens in the developer's head across two executions [19]. Diffing the reasoning structure between consecutive turns of a single conversation, reporting nodes added and invalidated and certainty shifts, and filtering to what the last message changed, appears to be novel. The comparison problem is the same graph-matching problem Technique 15 raises, and the two should share an implementation.

9. Conversation Spine

A thin persistent minimap represents the whole session as reasoning structure rather than message count. Decision points appear as marks, certainty as color, and human interventions as a distinct glyph. The spine is scroll-linked, and clicking a mark jumps to that point in the conversation.



The glyph choice carries weight for the claim that the user's judgment matters most at particular points. The spine makes the human's imprint on the session legible at a glance and gives a visual measure of how much of the reasoning the user touched. A session with two intervention glyphs across forty decision points looks different from one with twenty, and the user can see the difference without being told.

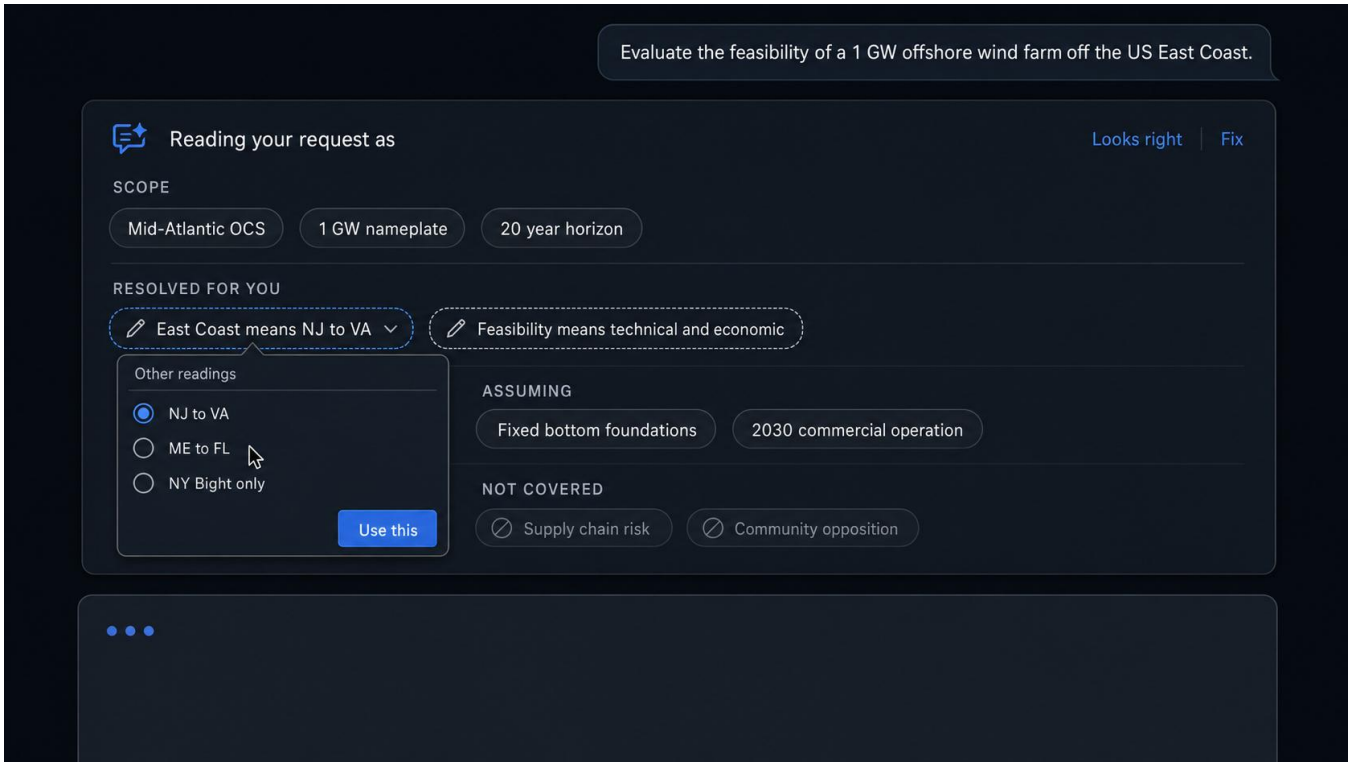
Minimaps for LLM conversations already ship, keyed to messages. Several browser extensions add a message-level minimap sidebar to ChatGPT with hover previews and click-to-jump, borrowing the pattern directly from code editors. In research, Sensecape externalizes levels of abstraction in a hierarchy view alongside a canvas and reports that users explored more topics with it [20], Graphologue converts responses into interactive node-link diagrams [21], and OnGoal supplies goal-progress summary views beside the transcript [13]. AGDebugger includes an overview visualization for navigating long agent histories [19].

Every one of those keys the overview to messages, nodes, insights, or goals. Keying it to reasoning structure, so that decision points become the marks and human interventions get their own glyph, is the part that supports the claim about where user judgment matters, and it is the part without precedent. The technique is a refinement of a shipping pattern rather than a new pattern, which is a reason to treat it as a supporting element rather than a study target.

2.4 Intervention Rather Than Inspection

10. Frame Card

Before the answer, a compact card restates the interpreted task: entities resolved, ambiguities the model settled without asking, constraints it assumed, items it treated as out of scope. Each line is editable in place. The user clicks a resolved ambiguity, sees the alternate reading, and swaps it before the model spends tokens on the wrong question.



The card is cheap to build, requires no additional inference beyond a short structured preamble, and attacks a large share of bad answers. A model that resolves "the client" to the wrong party, or assumes a budget constraint the user did not state, produces a fluent answer to a question the user did not ask. Catching that before generation costs the user a few seconds of reading.

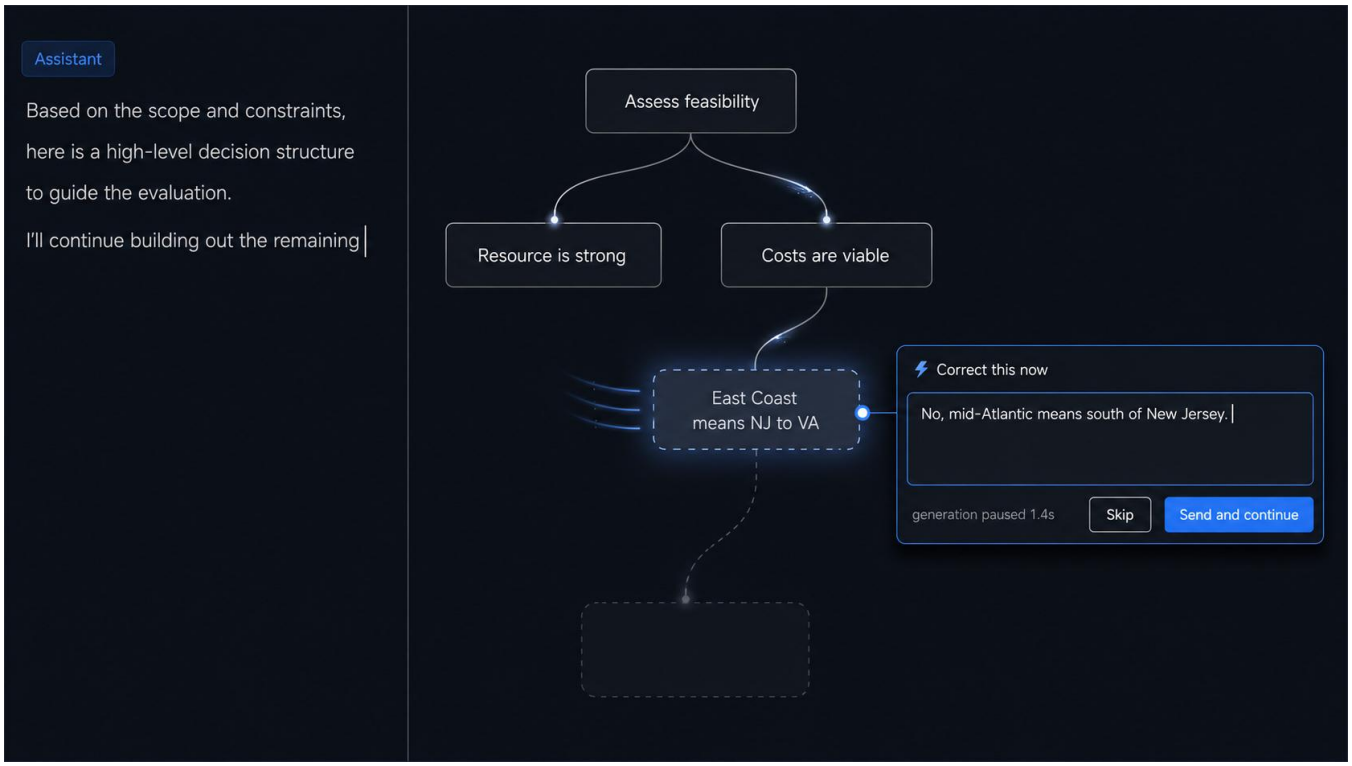
The design tension is interruption. A card the user must dismiss on every turn will be dismissed without reading by the third turn. Rendering it as a collapsed strip that expands on hover, with only settled ambiguities marked, keeps the cost proportional to the risk.

Coding tools ship this technique and general chat does not. Kiro built a product around it: the spec flow generates a requirements document from the user's prompt, asks clarifying questions about scope, constraints, and edge cases, and holds an approval gate before implementation, on the stated rationale that a deceptively simple prompt carries unstated assumptions that send the implementation in the wrong direction [14]. The plan modes in Claude Code and Cursor present an interpreted plan the user edits before execution. Deep research modes restate scope before committing to a long run. The pattern is well enough established in developer tooling to have a name, spec-driven development, and Kiro reports users trading roughly twenty minutes of spec review against debugging misaligned output later.

Two things are missing from all of it. None of these tools distinguishes the ambiguities the model settled without asking from the ones it surfaced, which is the specific class this card marks. And none applies the pattern to an ordinary chat turn; the approval gate is reserved for tasks the user already recognizes as large. The contribution is transferring a validated developer-tool pattern into general chat at a cost the user will tolerate on every turn, which makes the collapsed strip rendering the load-bearing design decision rather than a detail.

11. Mid-Stream Intervention Window

During generation, the interface renders the reasoning structure as it streams: the goal appears, hypotheses appear, evidence attaches to them. The user clicks a nascent node and injects a correction, and the model continues with the correction in context instead of restarting.



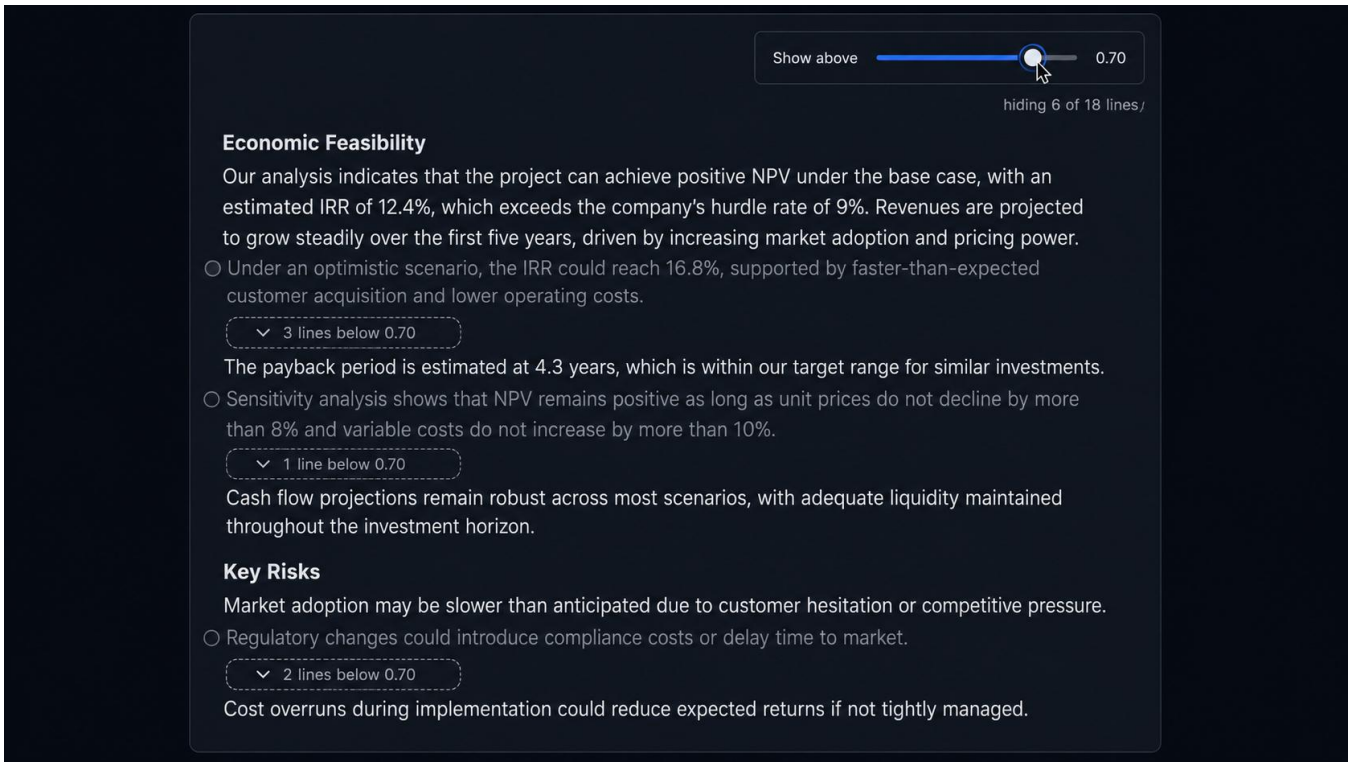
This technique addresses the promise the prior work's introduction makes and the prototypes do not keep. It also relocates a design question already scheduled for study with Prototype D into a setting where the interruption cost can be controlled: does the intervention window improve the answer enough to justify the attention it demands? Streaming intervention imposes a cost, since the user must attend during generation rather than after, and the answer may be that the cost exceeds the benefit for most tasks and falls below it for a specific class. Identifying that class is the contribution. **[TODO: confirm Prototype D's planned interruption-cost study and cite it]**

Intervention during generation exists at two granularities, and the reasoning node sits between them. HiLDe intervenes at the token: it pauses at the decision points where the model was least certain, shows the alternates with explanations of how each differs, and regenerates the remainder from the user's choice. In a within-subjects study with eighteen programmers on security tasks, participants produced significantly fewer vulnerabilities than with a conventional completion assistant [5]. AGDebugger intervenes at the message: users pause a running agent to send a message mid-run, or reset to an earlier point and edit a prior message, with agent state checkpointed so the reset is faithful. Its 14-participant study found the reset interaction central to how developers steer [19]. LangGraph provides the same capability in a runtime as interrupts and time travel, and the agentic coding assistants accept steering messages while a run is in flight.

HiLDe is the strongest existing evidence that mid-generation intervention improves output rather than merely satisfying a desire for control, and it also supplies a cost measurement: its authors report the interaction as deliberate slowing, accepted because the security payoff justified it. Streaming the reasoning structure and letting the user correct a nascent node is the untried granularity, and it inherits HiLDe's open question about which task classes repay the attention.

12. Certainty Threshold Rendering

Rather than placing a certainty bar beside the prose, restyle the prose itself. A slider sets a certainty floor. Text above the floor renders normally; text below it collapses to a marker the user can expand. At the top of the range the response shrinks to its defensible core.



This extends confidence scrubbing from the reasoning graph to the language, and it gives a clean study manipulation. Response length becomes a function of the threshold, which makes the relationship between certainty and content measurable for a given model and task: a response that collapses to two sentences at 80 percent certainty differs from one that retains half its length, and the difference is a property of the answer rather than the interface.

The risk is that collapsing low-certainty text removes the hedges that made the claim honest, leaving a confident-looking core that overstates. The expand affordance must stay visible enough that the user reads the core as a filtered view.

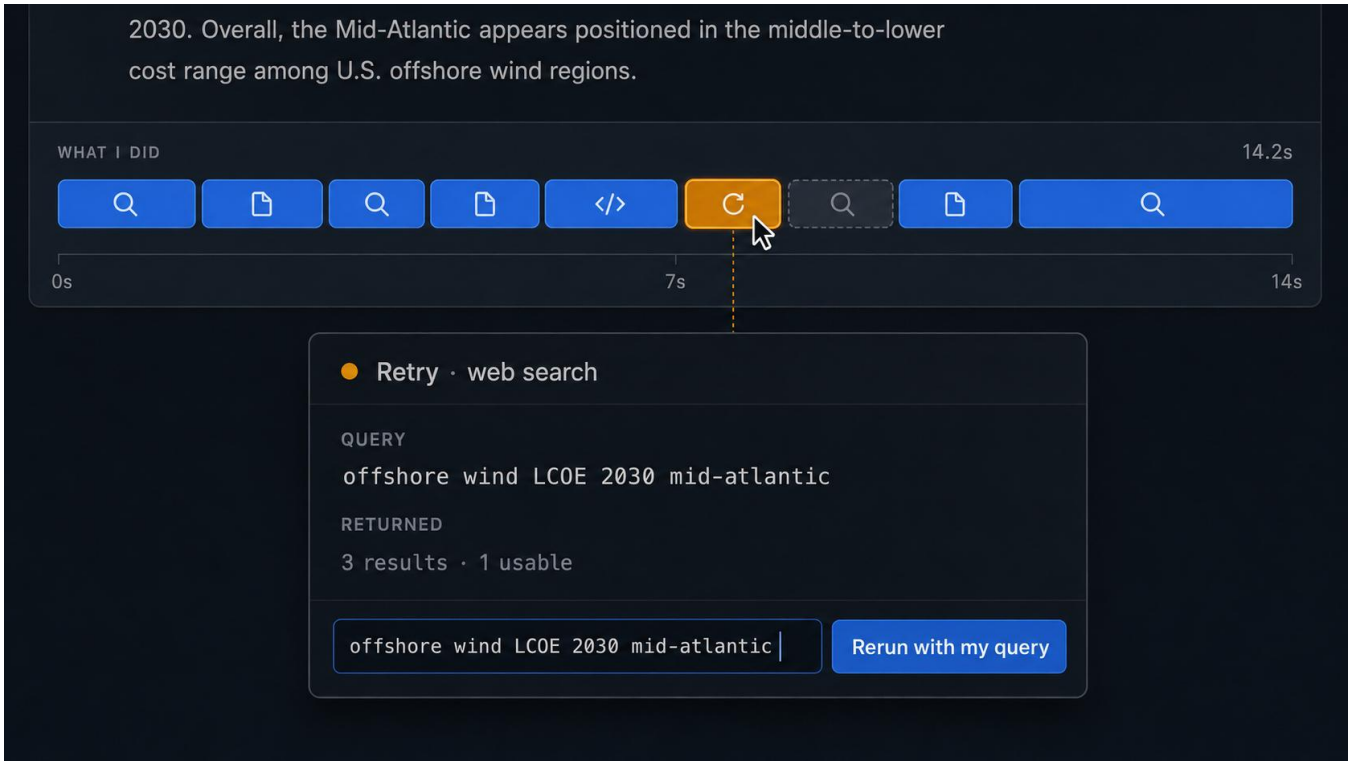
Threshold controls over uncertainty encodings exist; collapsing the prose does not. Uncertainty-highlighting interfaces commonly pair the encoding with a threshold so the user suppresses marks below a cutoff, which manages the density of the annotation without touching the text underneath. Progressive disclosure has an established record in transparency interfaces, where Springer and Whittaker found that withholding detail until the user asks for it changes how much scrutiny the user brings [22]. Response-length control shows up in products as a verbosity setting the user applies before generation.

Each of those filters the annotation, the disclosure, or the request. This technique filters the response itself, after generation, on a quantity the model reports per span. Restyling prose so that low-certainty text collapses to expandable markers, and treating the resulting length curve as a measurable property of the answer, appears to be novel. The novelty carries an obligation: no prior work establishes that users read a filtered core as filtered, so the study should measure whether they overstate the collapsed response rather than assume the expand affordance handles it.

2.5 Cross-Layer Traversal in a Chat Frame

13. Agent Action Ribbon

A horizontal micro-timeline sits under each response showing what the system did: searches issued, documents retrieved, code executed, retries, abandoned paths, each with its duration. The user clicks a segment for the query and the raw return.



The interaction that matters is query rewriting. A user who sees that the model searched for the wrong thing can fix the search and rerun the dependent reasoning without rewriting the prompt and losing the rest of the response. That operation is unavailable in current interfaces, which reduce the most accessible layer in the hierarchy to a spinner.

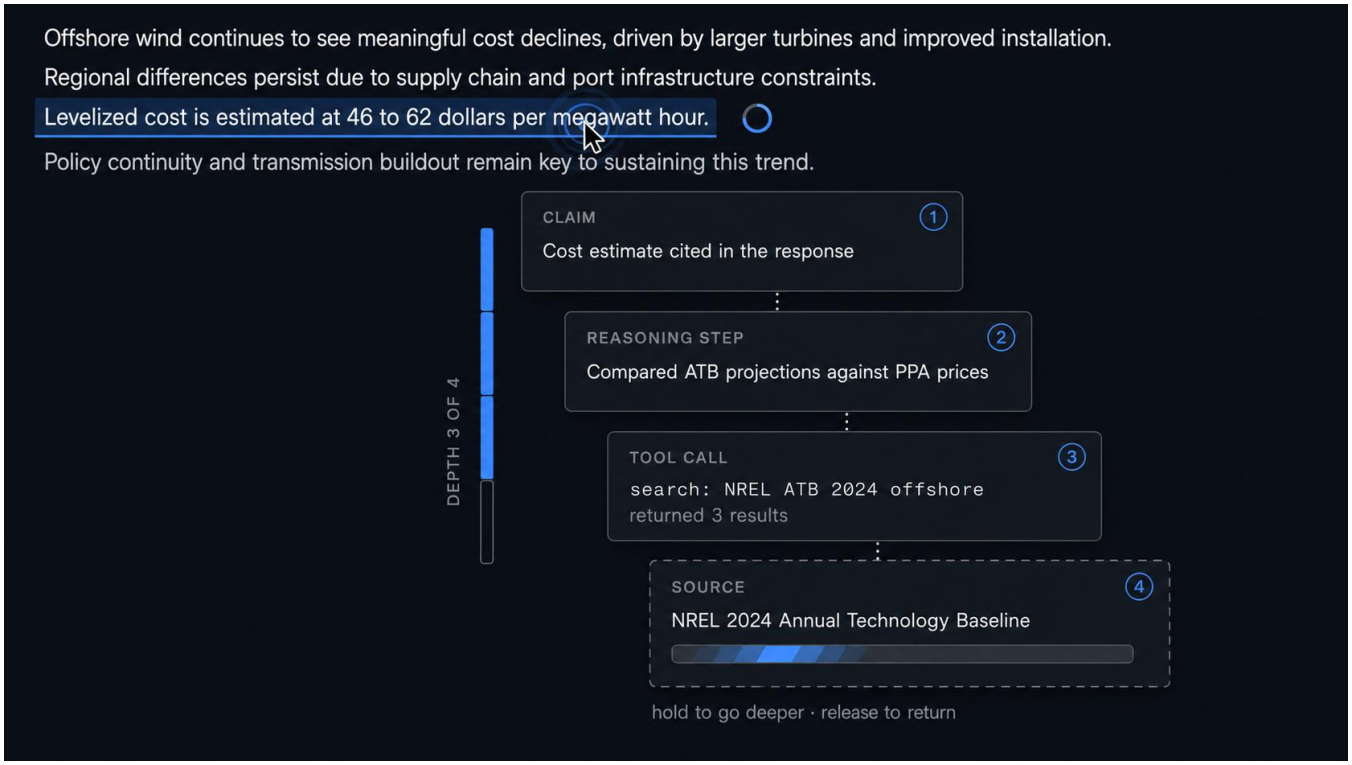
Agent actions are the easiest layer to instrument, since the orchestration code already logs them. The ribbon is largely a rendering problem rather than an access problem, which makes it a good early build.

This is the most widely deployed technique in the catalog. Perplexity shows the steps it took and the sources it pulled; the deep research modes in ChatGPT and Gemini render an activity panel of searches issued and pages read; the agentic coding assistants print each tool call and its return; and the tracing platforms give developers a full span timeline per run. In research, AgentLens organizes raw execution events into a behavior hierarchy, renders it as a hierarchical temporal visualization, and traces causes between an action and the memory or prior operation that produced it [23]. AGDebugger pairs its message viewer with an overview visualization for navigating long histories [19].

The rendering is solved, then, and the interaction is not. Every one of those displays is read-only with respect to the actions themselves: a user who sees the model searched for the wrong thing can restart the whole turn, and nothing more. Editing the query and rerunning only the reasoning that depended on it requires the dependency structure the rest of this catalog assumes, which is why the operation is absent from tools that otherwise render this layer well. Section 3 should treat the ribbon as a low-risk build whose contribution rests entirely on the rewrite interaction rather than on the timeline.

14. Provenance Descent

Press and hold any sentence and descend one layer per beat: the claim, then the reasoning step that produced it, then the tool call that fetched the evidence, then the source. Release to return to the surface.



The gesture keeps depth under continuous user control. A modal dialog commits the user to a context switch and forces an explicit return; press-and-hold makes descent reversible at any moment, so a user who wanted one layer and got two loses nothing. The author's reasoning lens spotlights supporting nodes at a single level; provenance descent walks the stack the six-layer model describes.

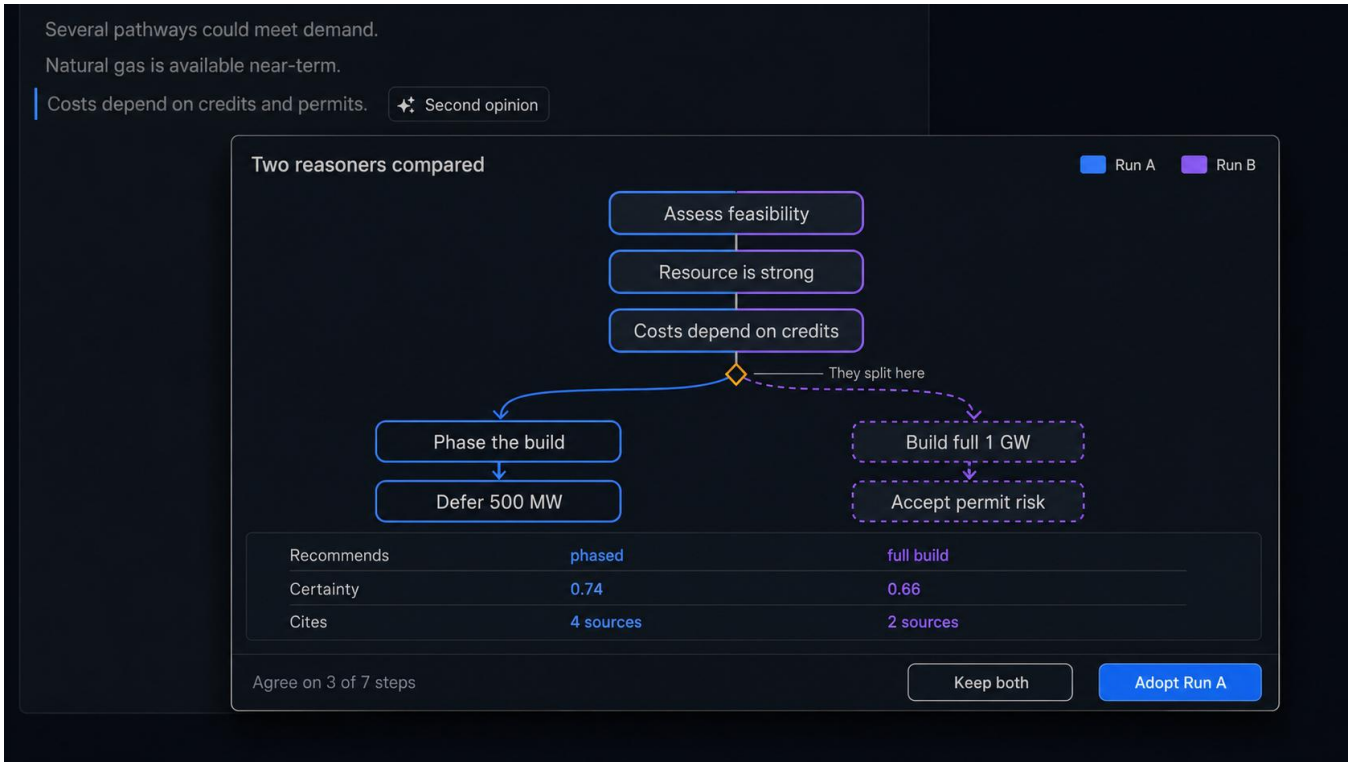
The gesture needs a desktop equivalent. Press-and-hold maps to touch; a click-and-hold or a modifier-scroll on the desktop preserves the continuous-control property. **[TODO: pick the desktop gesture and pilot it; the chat did not address non-touch input]**

Citation drill-down is a mature product pattern covering one hop of the descent. Perplexity attaches source chips to individual claims, previews the source on hover, and keeps a full source browser as a separate tab. NotebookLM links inline citations to the exact passage in the uploaded document. Anthropic's citations API returns character-level provenance into the source text, which gives implementers the granularity this technique needs at the bottom of the stack. The underlying principle is Shneiderman's details on demand, which the visual information-seeking mantra placed at the end of overview, zoom, and filter [24].

All of it stops at claim to source. The intermediate layers, the reasoning step that produced the claim and the tool call that fetched the evidence, are absent from every product surface I examined, and no interface treats depth as a continuous quantity the user controls with one sustained gesture. The novelty is the traversal rather than the endpoints. Since the existing pattern already trains users to expect a click on a citation to land on a source, the design risk is that press-and-hold competes with a learned interaction rather than extending it, and the pilot should test discoverability alongside the desktop gesture.

15. Second-Opinion Overlay

At a flagged node, the interface queries a second model, or the same model under a different framing, and renders the two reasonings aligned rather than side by side. Shared nodes merge. Divergent nodes fork, with the disagreement labeled. The user requests a second opinion on a specific claim instead of the whole answer, which keeps the cost proportional to the doubt.



Agreement between independent reasoners is a stronger trust signal than either one's self-report, since the failure modes that produce confident errors are partly model-specific. The alignment rendering carries the design weight. Two traces side by side leave the reader to perform the comparison; a merged graph performs it and shows the result, so the user reads the shape of the disagreement instead of reconstructing it.

Alignment across two reasoning traces is a graph-matching problem with no exact solution, and a wrong match reports agreement where none exists. Conservative matching, which merges only high-confidence correspondences and forks the rest, fails toward showing too much disagreement, which is the safer direction. **[TODO: specify the matching method and its confidence threshold]**

Querying a second reasoner is established and rendering the result as an alignment is not. Du et al. showed that multiple model instances proposing and critiquing each other's reasoning over several rounds produce more factually accurate answers than a single instance, which is the result this technique depends on [25]. On the interface side, LLM Comparator supports side-by-side analysis of two models' responses with an automatic judge, and its authors identify scalability and interpretability of the comparison as the central user difficulty [18]. ChainForge compares responses across models in a grid [11]. Public arenas and multi-model chat products let a user fire the same prompt at several models and read the answers in parallel.

Two departures matter. Every one of those comparisons operates on whole responses; this technique scopes the second opinion to a specific claim, which is what keeps the cost proportional to the doubt. And every one of them leaves the

comparison to the reader, which is the difficulty LLM Comparator's authors name. Merging shared nodes and forking divergent ones performs the comparison and shows its result. The alignment rendering is the contribution, and it is also the unsolved part; the graph-matching problem is the same one Technique 8 raises, so the two should share a matcher and a confidence threshold.

3. Implementation Priority and Constraints

3.1 Where the Value Concentrates

Techniques 2, 3, and 4 are the strongest publication candidates. Each converts a stated limitation in the author's prior work into a working interaction, and each depends on the self-hosted access the local deployment provides, which makes them difficult for groups building on hosted APIs to replicate. That difficulty is a contribution rather than an obstacle.

The survey in Section 2 revises that ranking. Techniques 2 and 3 hold: the agreement gutter has no precedent, and the knockout probe adapts a method the NLP community treats as routine into an operation no product exposes, which is a clean contribution to state. Technique 4 is the most crowded of the three, since ChainForge and Loom both put sampled distributions in front of users [11], [12]; its contribution narrows to in-chat rendering at the decision node, which is real but smaller than the other two. Techniques 8 and 12 have stronger novelty claims than 4 does. Both are also cheap: the reasoning diff needs no additional inference, and threshold rendering needs none either.

Techniques 6 and 10 are the cheapest to build and the most likely to show a large effect. Both attack framing and drift instead of presentation, and errors of framing and drift produce wrong answers rather than confusing ones. A study that measures answer quality, not subjective workload, should find them.

Prior art strengthens both of those bets rather than weakening them. OnGoal establishes that persistent session state reduces user time and effort in a controlled study [13], and Kiro establishes that a pre-generation frame is worth its review cost to developers [14]. Two independent results already point the direction the study would go, which lowers the risk of running it. The open questions become narrower and more answerable: whether drift detection catches contradictions the user would otherwise miss, and whether the frame card survives on a turn the user did not expect to review.

Technique 11 addresses the mid-stream correction that the prior work promises and the prototype set leaves unfulfilled. A preliminary examination committee reading the introduction against the prototypes may notice the gap, which is a reason to build the technique or to soften the promise. HiLDe gives the technique external support and a study design to borrow: token-level intervention during code generation reduced vulnerabilities against a conventional baseline with eighteen participants [5], so the interruption-cost question has an answer at one granularity and needs testing at another.

3.2 Inference Cost

Techniques 3, 4, 5, and 15 consume additional inference beyond the response itself. Sampled path distribution multiplies generation cost by the sample count; the knockout probe adds one generation per tested citation; the second-opinion overlay adds one generation per flagged node. Cost and latency become design constraints, and stating them at design time beats discovering them at build time.

Local deployment changes the arithmetic but does not remove it. GPU time is finite, and a technique that adds four seconds of latency to every response will be disabled by users regardless of what it costs the lab. The interfaces should treat additional inference as an on-demand operation the user initiates rather than a background enrichment, with the exception of resampling, which can run speculatively while the user reads the first response. **[TODO: measure the latency budget on the target hardware and set thresholds]**

3.3 The Encoding Budget

The visual channels in a chat frame are limited, and the design already spends most of them. Certainty, evidence credibility, and consequence weight must stay separable, and adding token entropy makes four quantities competing for color, opacity, texture, and position. **[TODO: confirm the three existing quantities against the prior papers; the chat referenced three without naming them]**

Two of the four can share a channel if they never appear together, and deciding which pairs are mutually exclusive is a design decision to settle before implementation. Density carries token entropy in Technique 1 for this reason: it leaves color to certainty. The gutter in Technique 2 avoids the problem by giving each layer its own position instead of encoding all of them in one mark. Techniques that resolve the budget by adding position are preferable to techniques that resolve it by adding a color scale.

4. Discussion

The fifteen techniques form a design space. Building all of them into one interface would produce an instrument that no user could read, and several pairs conflict directly: the hesitation ribbon and the agreement gutter both attach a continuous signal to the same text span, and certainty threshold rendering removes text that the provenance descent expects to be present. An implementation should select a coherent subset and justify the selection against a task.

Three properties distinguish the stronger candidates. They draw signal from a layer other than the model's self-report, they place a measurable operation under the user's control, and they produce a dependent variable a study can use. Techniques 3, 4, and 5 satisfy all three. Techniques 9 and 14 satisfy the second and offer navigation value without a clean measure, which makes them supporting elements rather than study targets.

The techniques also assume a reasoning structure the interface can address: nodes, dependencies, evidence attachments, decision points. Extracting that structure reliably from a model's output remains unsolved, and every technique inherits the extraction's error rate. An assumption ledger built on faulty extraction flags conflicts that are not conflicts and misses those that are. The extraction problem sits upstream of the interface work and deserves separate treatment. **[TODO: decide whether structure extraction belongs in this paper or a companion; the chat did not raise it]**

The survey supports a claim about where this work sits. The methods these techniques need are largely available: context ablation, entailment-based clustering of samples, token-level distributions, and multi-model disagreement all have published procedures and, in several cases, published discrimination results. What the methods lack is a place in a reading interface. Where an interaction does exist, it usually exists in a developer tool, and the pattern reappears often enough to be worth naming: HiLDe and AGDebugger give programmers mid-generation control, Kiro gives them a pre-generation frame, and the tracing platforms give them the action timeline. General chat users get a spinner and a collapsible panel. Treating the coding-assistant population as a testbed whose validated patterns transfer, rather than as a separate literature, is the cheapest route to several of the techniques above, and it also supplies study designs and effect sizes to plan against.

A caution attaches to that transfer. Programmers accept friction that general users may not, and the cost measurements those studies report were collected from participants whose task rewarded scrutiny. Technique 10's collapsed-strip rendering and Technique 12's expand affordance are both attempts to price the friction down for a reader rather than a reviewer, and neither has been tested on anyone.

5. Conclusion

Chat interfaces present model reasoning as prose and ask the user to judge it as prose. The fifteen techniques above give the user other things to read: the output distribution under the text, the agreement or disagreement among independent signals, the counts behind an alternative, the assumptions the session rests on, the actions the system took. Each technique attaches to

the chat frame rather than replacing it, which is the constraint that makes the set worth building. Users stay in the conversation; the inspection has to come to them.

References

[1] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," in *Advances in Neural Information Processing Systems 36*, 2023.

[2] H. Vasconcelos, G. Bansal, A. Fourney, Q. V. Liao, and J. W. Vaughan, "Generation Probabilities Are Not Enough: Uncertainty Highlighting in AI Code Completions," *ACM Transactions on Computer-Human Interaction*, vol. 32, no. 1, 2025, doi: 10.1145/3702320.

[3] Z. Buçinca, M. B. Malaya, and K. Z. Gajos, "To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making," *Proceedings of the ACM on Human-Computer Interaction*, vol. 5, no. CSCW1, Article 188, 2021, doi: 10.1145/3449287.

[4] S. Gehrmann, H. Strobelt, and A. M. Rush, "GLTR: Statistical Detection and Visualization of Generated Text," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 2019, pp. 111–116, doi: 10.18653/v1/P19-3019.

[5] E. Anaya González, R. Rothkopf, S. Lerner, and N. Polikarpova, "HiLDe: Intentional Code Generation via Human-in-the-Loop Decoding," in *Proceedings of the 2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2025.

[6] LM Studio, "Open Responses with local models via LM Studio,"

1. [Online]. Available: https://lmstudio.ai/blog/openresponses

[7] B. Cohen-Wang, H. Shah, K. Georgiev, and A. Mądry, "ContextCite: Attributing Model Generation to Context," in *Advances in Neural Information Processing Systems 37*, 2024, pp. 95764–95807.

[8] J. DeYoung, S. Jain, N. F. Rajani, E. Lehman, C. Xiong, R. Socher, and B. C. Wallace, "ERASER: A Benchmark to Evaluate Rationalized NLP Models," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4443–4458.

[9] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

[10] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting Hallucinations in Large Language Models Using Semantic Entropy," *Nature*, vol. 630, pp. 625–630, 2024.

[11] I. Arawjo, C. Swoopes, P. Vaithilingam, M. Wattenberg, and E. L. Glassman, "ChainForge: A Visual Toolkit for Prompt Engineering and LLM Hypothesis Testing," in *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, 2024, doi: 10.1145/3613904.3642016.

[12] Loom: Multiversal Tree Writing Interface for Human-AI Collaboration, 2020–. [Online]. Available: https://github.com/socketteer/loom

[13] A. J. Coscia, S. Guo, E. Koh, and A. Endert, "OnGoal: Tracking and Visualizing Conversational Goals in Multi-Turn Dialogue with Large Language Models," in *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*, Article 208, 2025, doi: 10.1145/3746059.3747746.

[14] Amazon Web Services, "Kiro Documentation: Specs," 2026. [Online]. Available: https://kiro.dev/docs/specs/

[15] M. Aliannejadi, H. Zamani, F. Crestani, and W. B. Croft, "Asking Clarifying Questions in Open-Domain Information-Seeking Conversations," in *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2019.

[16] M. J. Q. Zhang et al., "Modeling Future Conversation Turns to Teach LLMs to Ask Clarifying Questions," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[17] H. Strobelt, B. Hoover, A. Satyanarayan, and S. Gehrmann, "LMdiff: A Visual Diff Tool to Compare Language Models," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2021, pp. 96–105, doi: 10.18653/v1/2021.emnlp-demo.12.

[18] M. Kahng, I. Tenney, M. Pushkarna, M. X. Liu, J. Wexler, E. Reif, K. Kallarackal, M. Chang, M. Terry, and L. Dixon, "LLM Comparator: Visual Analytics for Side-by-Side Evaluation of Large Language Models," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*, 2024, doi: 10.1145/3613905.3650755.

[19] W. Epperson, G. Bansal, V. Dibia, A. Fourney, J. Gerrits, E. Zhu, and S. Amershi, "Interactive Debugging and Steering of Multi-Agent AI Systems," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*, 2025, doi: 10.1145/3706598.3713581.

[20] S. Suh, B. Min, S. Palani, and H. Xia, "Sensecape: Enabling Multilevel Exploration and Sensemaking with Large Language Models," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, 2023, doi: 10.1145/3586183.3606756.

[21] P. Jiang, J. Rayan, S. P. Dow, and H. Xia, "Graphologue: Exploring Large Language Model Responses with Interactive Diagrams," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, 2023, doi: 10.1145/3586183.3606737.

[22] A. Springer and S. Whittaker, "Progressive Disclosure: Empirically Motivated Approaches to Designing Effective Transparency," in *Proceedings of the 24th International Conference on Intelligent User Interfaces*, 2019, pp. 107–120.

[23] J. Lu, B. Pan, J. Chen, Y. Feng, J. Hu, Y. Peng, and W. Chen, "AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, 2025, doi: 10.1109/TVCG.2024.3394053.

[24] B. Shneiderman, "The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations," in *Proceedings of the 1996 IEEE Symposium on Visual Languages*, 1996, pp. 336–343.

[25] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving Factuality and Reasoning in Language Models through Multiagent Debate," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[TODO: add citations for the author's prior work referenced throughout: the certainty/confidence-scrubbing paper, the six-layer access paper, the prototype set (A through D), and the prompt taxonomy work]

[TODO: verify the DOI on reference [23] **and the page range on** [15] **against the publisher record before submission; both were taken from secondary citations rather than the source pages]**